Texo OCR Flutter SDK — Integration Guide

Oct 1, 2026 · @Akhil Saji

Overview

Texo is a Flutter SDK that uploads a document image to the **Texo** OCR API and returns the extracted text. You can drop in a ready-made button widget, or call the lower-level client when you need full control over the upload.

- Select or capture a document image on the device.
- Upload the image to the **Texo** OCR API.
- Show upload progress and manage loading state automatically.
- Retry transient server, network and timeout failures automatically.
- Handle failures through typed, sealed exceptions.
- Cancel an in-flight OCR request.
- Enforce the 10 MB upload limit before the image is uploaded.

Integration	Use when	Handles
TexoClientWidget	You want a ready-made OCR button	Picker, loading state, progress, upload, callbacks
ImageTextExtractor	You already have image bytes or need custom UI	OCR request, retries, progress, cancellation, typed errors

Installation

Add **Texo** to `pubspec.yaml`:

```
dependencies:
  texo: ^1.0.0
```

Then import the package:

```
import 'package:texo/texo.dart';
```

Getting your API key and document type

Purchase your API key and document type from the **Texo** dashboard at texo.klystron.app/dashboard. Both are tied to your account.

Value	Where to get it	Pass it to the SDK as
API key	Texo dashboard	`apiKey` (widget and client constructor), `apikey` (on `extractFromBytes`)
Document type	Texo dashboard, for example `drivinglicense`	`documentType`

Use the document type exactly as the dashboard shows it. A blank API key throws `ArgumentError`; an invalid one returns `UnauthorizedException`.

Quick start: drop-in widget

`TexoClientWidget` is a ready-to-use button. Tapping it opens the picker, uploads the selected image, manages loading and progress state, and returns either an `ExtractionResult` or an `ExtractionException`.

```
TexoClientWidget(
  label: 'Select document',
  icon: Icons.folder_open,
  apiKey: const String.fromEnvironment('TEXO_API_KEY'),
  documentType: 'drivinglicense',
  onImagePicked: (bytes, filename) =>
      setState(() => _preview = bytes),
  onResponse: (ExtractionResult result) {
    print(result.text);
    print(result.raw);
  },
  onError: (ExtractionException error) {
    print(error.message);
  },
)
```

- `result.text` — extracted text returned by the OCR service.
- `result.raw` — unmodified response body returned by the API.
- `onImagePicked` — the selected image bytes and filename, for preview or custom handling.

- `onError` — receives a typed `ExtractionException`.

Widget appearance and UX

Parameter	Default	Purpose
backgroundColor / foregroundColor	Theme primary / onPrimary	Button colors
icon	None	Leading IconData
showImageIcon + image	false	Use an ImageProvider as the leading icon
leadingSize	20	Icon and progress spinner size
padding	16 × 12	Inner button padding
borderRadius	20	Corner radius
enabled	true	Enable or disable the button; uploading disables it

If `onError` is not provided, failures show in a SnackBar. This needs a `Scaffold` ancestor.

Quick start: direct client

Use `ImageTextExtractor` when the app already has image bytes or needs its own selection and upload flow.

```
final extractor = ImageTextExtractor(apiKey: apiKey);

try {
  final result = await extractor.extractFromBytes(
    bytes,
    apikey: apiKey,
    documentType: 'drivinglicense',
    onProgress: (sent, total) => print('${sent / total}'),
  );

  print(result.text);
  print(result.prettyJson);
} on ExtractionException catch (e) {
  print(e.message);
} finally {
  extractor.close();
}
```

Always call `close()` when the extractor is no longer needed; it releases the underlying HTTP client. To cancel, pass a `CancelToken` and call `token.cancel()`; the request then throws `CancelledException`.

Client configuration

Parameter	Default	Purpose
apiKey	Required	API authentication; a blank value throws ArgumentError
baseUrl	Texo production API	Point at a different environment or endpoint
timeout	60 seconds	Receive timeout; the connect timeout is fixed at 15 seconds
maxRetries	2	Retries transient failures only

Error handling

Every failure is a subtype of the sealed `ExtractionException`, so the Dart compiler checks that a `switch` covers them all.

Exception	Scenario	Retried
BadRequestException	HTTP 400, 413, 422, or HTTP 200 with `success: false`	No
UnauthorizedException	HTTP 401 or 403; invalid or missing API key	No
FileTooLargeException	Image exceeds 10 MB; caught before upload	No
UnsupportedFormatException	Unknown image bytes or HTTP 415	No
RateLimitException	HTTP 429	No
ServerException	HTTP 5xx	Yes
NetworkException	Connection failure	Yes
TimeoutException	Connect, send or receive timeout	Yes
CancelledException	CancelToken cancelled	No
PickerException	Picker or camera failed, or the file could not be read	No; widget only
UnknownException	Unexpected failure	No

Retries use exponential backoff of 400 ms, 800 ms and 1600 ms.

```
onError: (error) => setState(() => _message = switch (error) {
  UnauthorizedException() => 'Check your API key.',
  PickerException() => 'Allow camera access to continue.',
  FileTooLargeException(:final limit) =>
      'Keep images under ${limit ~/ 1024} KB.',
  _ => error.message,
}),
```

Upload limits and supported formats

- Maximum upload size is 10 MB, enforced on the device before upload.
- Image type is detected from magic bytes, not the filename.
- Supported formats: JPEG, PNG, GIF, BMP, WebP and HEIC.

Platform setup

Only the camera needs platform permission configuration. Files and gallery use `file_picker` and need no extra permissions on iOS, Android, Web or Windows.

iOS — camera

Add `NSCameraUsageDescription` to `ios/Runner/Info.plist`. Without it, opening the camera can terminate the app.

```
<key>NSCameraUsageDescription</key>
<string>Used to photograph documents for text extraction.</string>
```

`NSPhotoLibraryUsageDescription` is not needed, because the gallery uses PHPicker.

Android, Web and Windows

No Texo-specific configuration is needed. Android requires compileSdk 34 or higher.

macOS

For sandboxed apps, add the user-selected read-only files entitlement to both `DebugProfile.entitlements` and `Release.entitlements`.

```
<key>com.apple.security.files.user-selected.read-only</key>
<true/>
```

Example app

A complete runnable example is in the `example/` directory of the Texo repository.

```
cd example
flutter run --dart-define=TEXO_API_KEY=your_key_here
```

Integration checklist

- [ ] API key purchased from the Texo dashboard and passed through `--dart-define` or secure app configuration.
- [ ] Document type purchased from the Texo dashboard and matches `documentType` exactly.
- [ ] A sample document image is ready for testing.
- [ ] The app has network connectivity.

- [ ] iOS camera: `NSCameraUsageDescription` is configured.
- [ ] macOS: the sandbox file entitlement is configured.